

TUGAS PRAKTIKUM

JARINGAN SYARAF TIRUAN

Nama	:	Wakhid Nugroho
NIM	:	H1D024003
Shift KRS	:	F
Shift Baru	:	F
Link Repositori Github	:	https://github.com/wakhid3273/H1D024003-PraktikumKB-Pertemuan6

Penjelasan Source Code pada setiap file

File: Perceptron.py

```
# import library
import numpy as np
import matplotlib.pyplot as plt

# buat kelas perceptron
class Perceptron:
    # Simpan learning rate dan max epoch dalam konstruktor
    def __init__(self, alpha=0.1, epoch=10):
        self.alpha = alpha
        self.epoch = epoch

    # Fungsi menghitung nilai y_in atau net
    def weighted_sum(self, X):
        return np.dot(X, self.w_[1:]) + self.w_[0]

    # Fungsi menerapkan fungsi aktivasi bipolar
    def predict(self, X):
        return np.where(self.weighted_sum(X) >= 0.0, 1, -1)
```

```

# Fungsi membuat simulasi garis pemisah data
def plot_decision_boundary(self, X, t, epoch):
    # Membuat titik data input
    plt.scatter(X[:, 0], X[:, 1], c=t.ravel(), marker='o', edgecolors='k', cmap=plt.cm.RdYlBu)

    # Menentukan limit tampilan bidang grafik
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1

    # Membuat garis pemisah
    x_vals = np.linspace(x_min, x_max, 100)
    y_vals = -(self.w_[0] + self.w_[1] * x_vals) / self.w_[2]
    plt.plot(x_vals, y_vals, 'b-', label=f'Decision boundary (Epoch {epoch+1})')

    plt.xlim(x_min, x_max)
    plt.ylim(y_min, y_max)
    plt.title(f'Decision Boundary Pada Epoch {epoch+1}')
    plt.xlabel('X1')
    plt.ylabel('X2')
    plt.legend()
    plt.show()

# Fungsi utama Perceptron
def fit(self, X, t):
    # Inisialisasi bobot dan bias awal = 0
    self.w_ = np.zeros(1 + X.shape[1])

    # Menyimpan hasil pada HasilPerceptron.txt
    with open("HasilPerceptron.txt", "w") as f:

```

```

f.write("Masalah OR dengan Perceptron\n")
f.write("-----\n")
f.write(f"Input : \n{X}\n")
f.write(f"Target: \n{t}\n")
f.write(f"Bobot awal : {self.w_[1:]}\n")
f.write(f"Bias awal : {self.w_[0]}\n")
f.write(f"Learning rate : {self.alpha}\n")
f.write(f"Max Epoch : {self.epoch}\n")

# Iterasi Perceptron(Epoch)
for epoch in range(self.epoch):
    f.write(f"\nEpoch {epoch + 1}/{self.epoch}\n")
    f.write("-----\n")
    error = np.array([])

    # Iterasi setiap pasang matriks input dengan targetnya
    for xi, target in zip(X, t):
        # Periksa input dengan model Perceptron
        y_pred = self.predict(xi)

        # Periksa apakah output sesuai target, jika iya maka error = 0, bobot tetap
        error = np.append(error, target - y_pred)

        # Modifikasi bobot dengan Delta Rule, jika error = 0 maka update = 0
        update = self.alpha * error[-1]

        # Modifikasi bobot w
        self.w_[1:] += update * xi

        # Modifikasi bias b
        self.w_[0] += update

```

```

# Menuliskan hasil tiap iterasi input pada satu epoch

f.write(f'Input: {xi}, Target: {target}, Predict: {y_pred}, Error: {error[-1]},
Bobot: {self.w_[1:]}, Bias: {self.w_[0]}\n')

# Simulasikan garis pemisah model Perceptron
self.plot_decision_boundary(X, t, epoch)

# Menuliskan penjumlahan kuadrat error setiap input
f.write(f'Sum Square Error(SSE): {sum(error ** 2)}\n')

# Periksa kondisi berhenti, jika penjumlahan kuadrat error setiap input = 0 atau
max epoch tercapai
if sum(error ** 2) == 0 or epoch + 1 == self.epoch:
    f.write("-----\n")
    f.write(f'Pelatihan berhenti pada epoch ke-{epoch + 1} karena ')
    f.write("Sum Square Error(SSE) mencapai target.\n" if epoch + 1 != self.epoch
else "max epoch tercapai.\n")

# Menuliskan bobot terakhir
f.write(f'\nBobot akhir :{self.w_[1:]}\n')
f.write(f'Bias akhir :{self.w_[0]}\n')
break

```

A. Struktur Kelas Perceptron

- `__init__(self, alpha=0.1, epoch=10)`: Konstruktor yang menyimpan learning rate (alpha) dan jumlah maksimum epoch.
- `weighted_sum(self, X)`: Menghitung nilai net input (y_{in}) menggunakan dot product antara input X dengan bobot, ditambah bias.
- `predict(self, X)`: Menerapkan fungsi aktivasi bipolar. Output = 1 jika $net \geq 0$, dan -1 jika $net < 0$.
- `plot_decision_boundary(self, X, t, epoch)`: Membuat visualisasi scatter plot data input beserta garis batas keputusan (decision boundary) pada setiap epoch.

- `fit(self, X, t)`: Fungsi utama pelatihan. Melakukan iterasi epoch, update bobot menggunakan Delta Rule, dan menyimpan log ke file HasilPerceptron.txt.

B. Algoritma Pelatihan

Langkah-langkah yang dilakukan dalam fungsi `fit`:

- Inisialisasi semua bobot dan bias bernilai 0.
- Untuk setiap epoch, iterasi setiap pasang data input-target.
- Hitung prediksi menggunakan model saat ini.
- Hitung $\text{error} = \text{target} - \text{prediksi}$.
- Update bobot: $w = w + \alpha * \text{error} * x_i$
- Update bias: $b = b + \alpha * \text{error}$
- Hitung Sum Square Error (SSE). Jika $\text{SSE} = 0$ atau epoch maksimum tercapai, pelatihan berhenti.
- Setiap epoch, simpan log detail ke HasilPerceptron.txt.

File: Perceptron_or.py

```
# Import library

import numpy as np
import Perceptron as p

# Inisialisasi input dan target
X = np.array([[1,1], [1,-1], [-1,1], [-1,-1]])
t = np.array([[1], [1], [1], [-1]])

# Pemanggilan model Perceptron
model = p.Perceptron(alpha=0.1, epoch=10)
model.fit(X, t)
```

A. Data Input dan Target

Data yang digunakan adalah tabel kebenaran fungsi OR dengan representasi bipolar (+1 / -1):

X1	X2	Target
1	1	1
1	-1	1

-1	1	1
-1	-1	-1

Hanya pasang input (-1, -1) yang menghasilkan output -1 (False), sedangkan kombinasi lainnya menghasilkan 1 (True), sesuai dengan definisi logika OR.

B. Konfigurasi Model

- Learning rate (alpha): 0.1
- Jumlah maksimum epoch: 10

File: Backpropagation.py

```
# Import library

import numpy as np
import matplotlib.pyplot as plt

# Buat kelas Backpropagation
class Backpropagation:

    # Simpan learning rate, epoch, dan target error dalam konstruktor
    # serta inisialisasi bobot dan bias awal random
    def __init__(self, alpha, epoch, target_error):
        self.alpha = alpha
        self.epoch = epoch
        self.target_error = target_error
        self.n_input = 2
        self.n_hidden = 2
        self.n_output = 1
        self.w_hidden = np.random.rand(self.n_input, self.n_hidden)
        self.b_hidden = np.random.rand(1, self.n_hidden)
        self.w_output = np.random.rand(self.n_hidden, self.n_output)
        self.b_output = np.random.rand(1, self.n_output)

    # Fungsi aktivasi sigmoid bipolar (tanh), output: (-1, 1)
    def bi_sigmoid(self, x):
```

```

return np.tanh(x)

# Turunan fungsi aktivasi tanh (asumsi x = output tanh)
def deriv_bi_sigmoid(self, x):
    return 1 - x**2

# Fungsi visualisasi penurunan error per epoch
def plot_error(self, x, epoch):
    plt.plot(range(1, epoch + 1), x, linestyle='-', color='b', label='Error')
    final_error = x[-1]
    plt.annotate(
        f'Epoch {epoch}, Error: {final_error:.4f}',
        xy=(epoch, final_error),
        xytext=(epoch - len(x) * 0.2, final_error + 0.05),
        arrowprops=dict(facecolor='black', arrowstyle='->'),
        fontsize=10,
        color='red'
    )
    plt.title('Perbaikan Error Setiap Epoch')
    plt.xlabel('Epoch')
    plt.ylabel('Sum Square Error(SSE)')
    plt.grid(True)
    plt.legend()
    plt.show()

# Fungsi utama Backpropagation
def fit(self, X, t):
    errors_per_epoch = []

    with open("hasilBackpropagation.txt", "w") as f:

```

```

f.write("Masalah XOR dengan Backpropagation\n")
f.write("-----\n")
f.write(f"Input : \n{X}\n")
f.write(f"Target : \n{t}\n\n")
f.write(f"Bobot awal hidden layer : \n{self.w_hidden}\n\n")
f.write(f"Bias awal hidden layer : \n{self.b_hidden}\n\n")
f.write(f"Bobot awal output layer : \n{self.w_output}\n\n")
f.write(f"Bias awal output layer : \n{self.b_output}\n\n")
f.write(f"Learning rate : {self.alpha}\n")
f.write(f"Max Epoch   : {self.epoch}\n\n")

```

```

for epoch in range(self.epoch):

```

```

    f.write("-----\n")

```

```

    f.write(f"Epoch {epoch + 1}/{self.epoch}\n")

```

```

    f.write("-----\n")

```

```

    total_error = 0

```

```

    count = 1

```

```

    output = np.array([])

```

```

    for xi, target in zip(X, t):

```

```

        f.write(f>Data ke-{count}\n")

```

```

        f.write("-----\n")

```

```

        f.write("----- Forward Propagation -----\n")

```

```

        # Input -> Hidden

```

```

        h_in = np.dot(xi, self.w_hidden) + self.b_hidden

```

```

        f.write(f"Operasi input ke hidden layer: \n{h_in}\n\n")

```

```

        h = self.bi_sigmoid(h_in)

```

```

        f.write(f"Aktivasi hidden layer: \n{h}\n\n")

```



```

# Hidden -> Output
y_in = np.dot(h, self.w_output) + self.b_output
f.write(f'Operasi hidden ke output layer:\n{y_in}\n\n')
y = self.bi_sigmoid(y_in)
output = np.append(output, y)
f.write(f'Aktivasi output layer:\n{y}\n')

f.write("----- Backward Propagation ----- \n")

# Error dan delta output layer
error = target - y
total_error += np.sum(error**2)
f.write(f'Error:\n{error}\n\n')

d_y = error * self.deriv_bi_sigmoid(y)
f.write(f'Delta output (d_y):\n{d_y}\n\n')

# Error dan delta hidden layer
error_h = np.dot(d_y, self.w_output.T)
f.write(f'Error hidden layer (error_h):\n{error_h}\n\n')

d_h = error_h * self.deriv_bi_sigmoid(h)
f.write(f'Delta hidden layer (d_h):\n{d_h}\n\n')

# Update bobot dan bias output layer
self.w_output += np.dot(h.T, d_y) * self.alpha
f.write(f'Bobot output layer baru (w_output):\n{self.w_output}\n\n')
self.b_output += np.sum(d_y, axis=0, keepdims=True) * self.alpha
f.write(f'Bias output layer baru (b_output):\n{self.b_output}\n\n')

```

```

# Update bobot dan bias hidden layer
self.w_hidden += np.dot(xi.reshape(2, 1), d_h) * self.alpha
f.write(f"Bobot hidden layer baru (w_hidden):\n{self.w_hidden}\n\n")
self.b_hidden += np.sum(d_h, axis=0, keepdims=True) * self.alpha
f.write(f"Bias hidden layer baru (b_hidden):\n{self.b_hidden}\n")
f.write("-----\n")

count += 1

# Hitung average SSE per epoch
average_error = total_error / len(X)
errors_per_epoch.append(average_error)
f.write(f"Output : {output.reshape(1, 4)}\n")
f.write(f"Sum Square Error(SSE) epoch ke-{epoch + 1}: {average_error}\n")

# Cek kondisi berhenti
if average_error < self.target_error or epoch + 1 == self.epoch:
    f.write("-----\n")
    f.write(f"Pelatihan berhenti pada epoch ke-{epoch + 1} karena ")
    f.write(
        "Sum Square Error(SSE) mencapai target.\n"
        if average_error < self.target_error
        else "max epoch tercapai.\n"
    )
    f.write(f"\nBobot akhir hidden layer :\n{self.w_hidden}\n\n")
    f.write(f"Bias akhir hidden layer :\n{self.b_hidden}\n\n")
    f.write(f"Bobot akhir output layer :\n{self.w_output}\n\n")
    f.write(f"Bias akhir output layer :\n{self.b_output}\n\n")
    self.plot_error(errors_per_epoch, epoch + 1)
    break

```

A. Arsitektur Jaringan

- Input layer: 2 neuron (X1, X2)
- Hidden layer: 2 neuron dengan fungsi aktivasi tanh
- Output layer: 1 neuron dengan fungsi aktivasi tanh

B. Konstruktor `__init__`

Menyimpan hyperparameter dan menginisialisasi bobot serta bias secara acak menggunakan `np.random.rand`:

- `alpha`: learning rate (laju pembelajaran)
- `epoch`: jumlah maksimum epoch
- `target_error`: nilai SSE yang menjadi kondisi berhenti
- `w_hidden`: bobot dari input ke hidden layer, bentuk (2, 2), nilai acak
- `b_hidden`: bias hidden layer, bentuk (1, 2), nilai acak
- `w_output`: bobot dari hidden ke output layer, bentuk (2, 1), nilai acak
- `b_output`: bias output layer, bentuk (1, 1), nilai acak

C. Fungsi `bi_sigmoid` (Sigmoid Bipolar / tanh)

Menerapkan fungsi aktivasi tanh (hyperbolic tangent) sebagai fungsi aktivasi sigmoid bipolar. Output berada di rentang (-1, +1), cocok untuk representasi bipolar.

D. Fungsi `deriv_bi_sigmoid` (Turunan tanh)

Menghitung turunan dari fungsi tanh. Karena argumen `x` diasumsikan sudah merupakan output tanh, turunannya dihitung sebagai $1 - x^2$ (rumus turunan tanh yang efisien).

E. Fungsi `plot_error`

Membuat visualisasi grafik penurunan Sum Square Error (SSE) per epoch menggunakan Matplotlib. Menampilkan nilai SSE akhir beserta nomor epoch dengan anotasi panah merah pada titik akhir kurva.

File `Backpropagation_xor.py`

```
# Import library
```

```
import numpy as np
```

```
import Backpropagation as b
```

```
# Inisialisasi input dan target XOR bipolar
```

```
X = np.array([[1, 1], [1, -1], [-1, 1], [-1, -1]])
```

```
t = np.array([[-1], [1], [1], [-1]])
```

Pemanggilan model Backpropagation

```
model = b.Backpropagation(alpha=0.3, epoch=1000, target_error=0.001)
```

```
model.fit(X, t)
```

A. Data Input dan Target XOR Bipolar

Tabel kebenaran XOR dengan representasi bipolar (+1 = True, -1 = False):

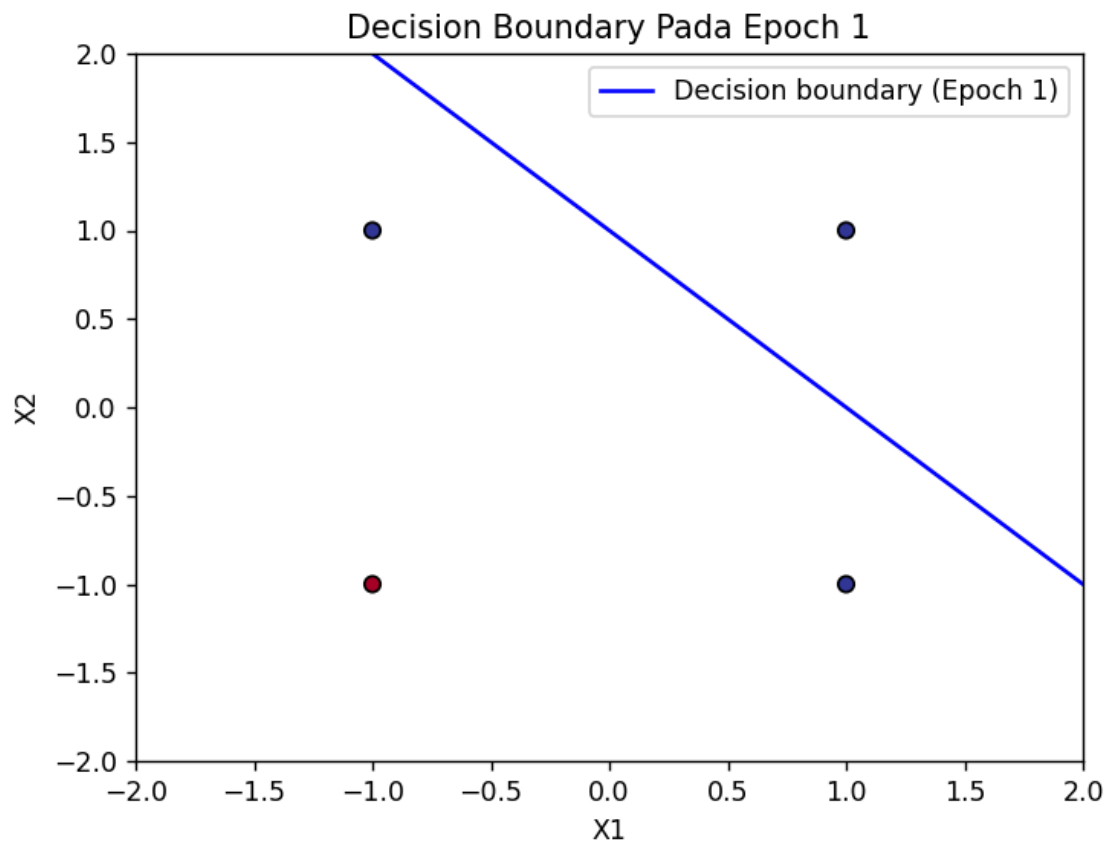
X1	X2	Target (XOR)
1	1	-1
1	-1	1
-1	1	1
-1	-1	-1

XOR menghasilkan True (+1) hanya jika kedua input berbeda, dan False (-1) jika kedua input sama. Ini berbeda dengan OR bipolar, dan tidak dapat dipisahkan dengan satu garis lurus (non-linearly separable).

B. Konfigurasi Model

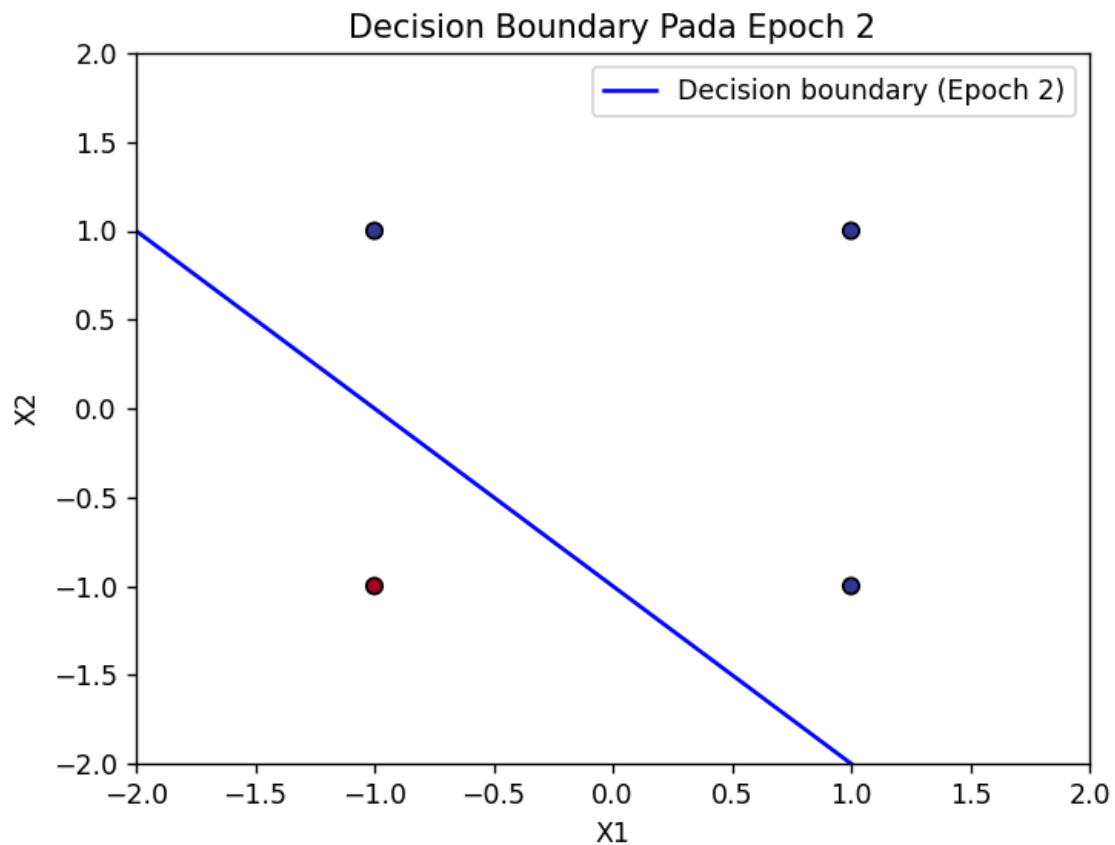
- Learning rate (alpha): 0.3
- Jumlah maksimum epoch: 1000
- Target error (SSE): 0.001
- Model berhenti lebih awal jika $SSE < 0.001$ sebelum epoch 1000

Penjelasan Grafik



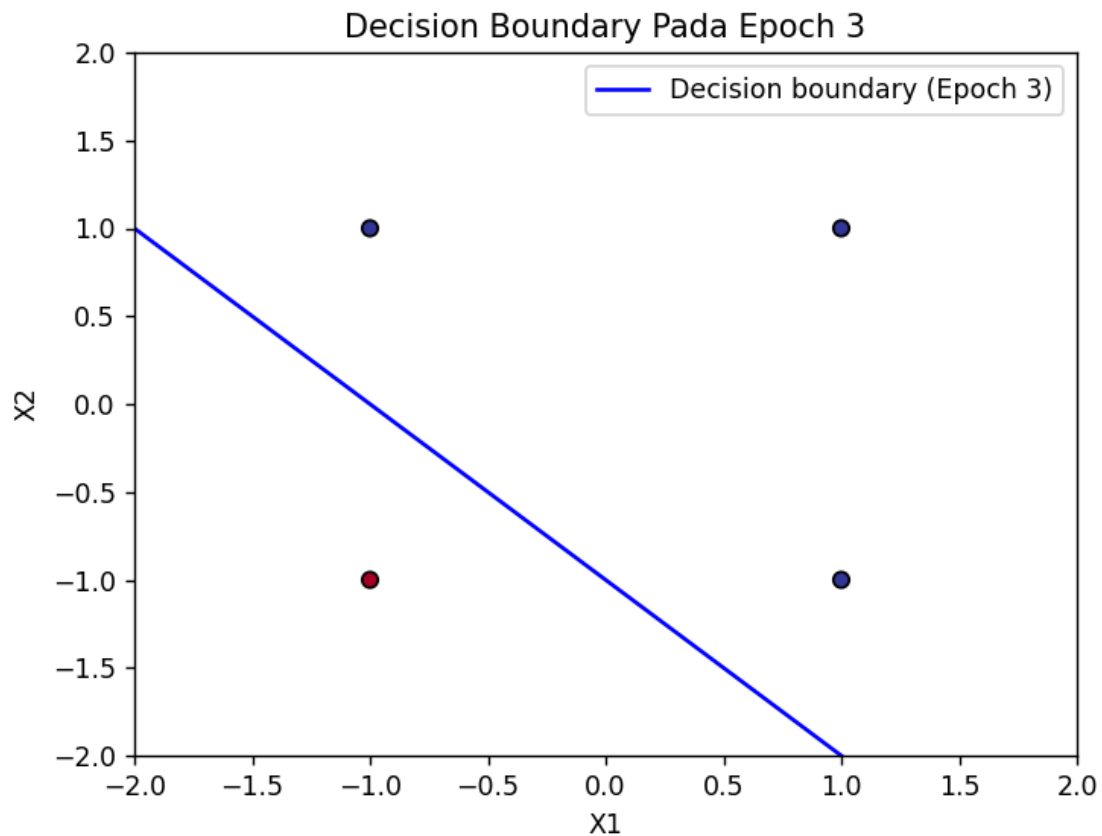
Pada epoch pertama, garis batas keputusan terbentuk dengan posisi miring dari atas-kiri menuju bawah-kanan. Garis ini belum mampu memisahkan data secara sempurna. Terlihat bahwa titik berwarna merah (kelas -1, input [-1,-1]) masih berada di sisi yang sama dengan sebagian titik biru (kelas +1). Hal ini menandakan bahwa bobot masih dalam proses penyesuaian dan model belum konvergen.

Pada epoch ini, model menerima semua data latih satu per satu, menghitung prediksi, dan memperbarui bobot menggunakan Delta Rule. Karena dimulai dari bobot nol, perubahan bobot pada epoch awal cukup signifikan.

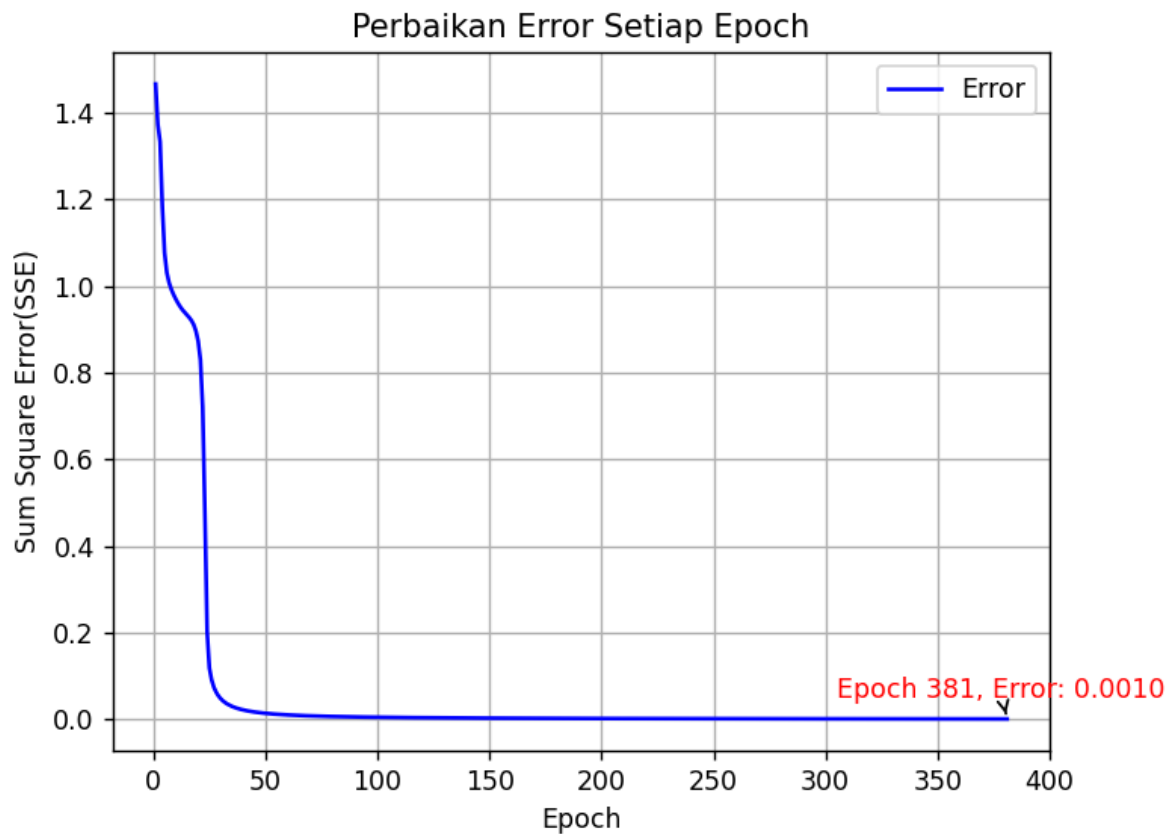


Pada epoch kedua, posisi garis pemisah telah bergeser. Garis kini tampak lebih miring ke kiri dan menyentuh batas bawah grafik di sekitar $X_1 = 1$, $X_2 = -2$. Dibandingkan epoch 1, pemisahan data sudah lebih baik — titik merah $(-1, -1)$ kini mulai berada di bawah garis pemisah, sementara ketiga titik biru berada di atas garis.

Perubahan posisi garis ini merupakan hasil dari update bobot pada epoch sebelumnya. Model terus belajar dari kesalahan yang terjadi pada setiap iterasi data.



Pada epoch ketiga, posisi garis pemisah hampir identik dengan epoch 2. Ini mengindikasikan bahwa bobot sudah mendekati nilai konvergen. Titik merah (kelas -1) sudah berada di sisi yang benar (di bawah garis), sementara semua titik biru (kelas +1) berada di atas garis pemisah. Kestabilan garis dari epoch 2 ke epoch 3 menunjukkan bahwa Sum Square Error (SSE) sudah sangat kecil atau mendekati nol, dan model hampir mencapai kondisi konvergen. Pelatihan berhenti saat $SSE = 0$ atau jumlah epoch maksimum (10) tercapai.



Grafik ini menampilkan perubahan nilai Sum Square Error (SSE) selama proses pelatihan Backpropagation pada masalah XOR bipolar sepanjang 381 epoch.

Terdapat tiga fase yang terlihat jelas pada grafik:

- **Fase awal (Epoch 1–5):** SSE berada di titik tertinggi sekitar 1.45. Hal ini wajar karena bobot dan bias diinisialisasi secara acak sehingga belum teroptimasi untuk masalah XOR, menghasilkan prediksi yang jauh dari target.
- **Fase penurunan tajam (Epoch 5–50):** SSE turun dengan sangat cepat mendekati nol. Ini adalah fase di mana jaringan saraf belajar paling efektif — delta error yang besar menyebabkan bobot diperbarui secara signifikan melalui mekanisme backpropagation, dan model mulai memahami pola XOR bipolar.
- **Fase konvergen (Epoch 50–381):** SSE mendatar sangat dekat dengan nol dan stabil. Perubahan bobot per iterasi menjadi sangat kecil karena delta error sudah mendekati nol, menandakan model telah berhasil mempelajari fungsi XOR. Nilai SSE akhir pada epoch ke-381 adalah 0.0010, yang ditandai dengan anotasi merah pada grafik. Nilai ini menunjukkan bahwa model telah berhasil meminimalkan error hingga mendekati nol, sehingga jaringan dapat memprediksi keluaran XOR bipolar

dengan sangat akurat. Pelatihan berhenti lebih awal sebelum batas maksimum 1000 epoch karena kondisi $\text{target_error} < 0.001$ telah terpenuhi.